KUBRIC REACH — Architecture Diagram

Internal Developer Platform · Zoom Integration · v1.0

CLIENT LAYER

Browser App
Lovable-Hosted Next.js
GitHub OAuth SSO
TLS 1.2+ Encrypted

Five-Zone UI Shell (h-screen overflow-hidden)
- Activity Bar 52px
- Left Panel 240px
- Center Monaco IDE / Kanban flex
- Right Panel 300px

VS Code Extension
Issues TreeView
Commit Webview
GitHub OAuth Token

MCP Server
Developer Self-Hosted
Docker Compose
ws_xxxx API Key Auth

Zoom Meeting
Volunteer Contributor
OBF / ZAK Tokens
TLS Encrypted

API / EDGE FUNCTION LAYER — Next.js App Router Route Handlers (verify session → check role → inject workspace_id)

POST /api/github/proxy
🔒 Security Boundary #1
All GitHub calls proxied here
provider_token server-side only
SSRF: endpoint must start with /
Absolute URLs rejected (403)
Never exposes token to browser

Issues API
GET/POST /api/issues
PATCH /api/issues/[id]
GitHub = Master Record
Supabase = Cache Layer
GitHub write FIRST always
Pusher broadcast on change
Notion sync on status done
Zod-validated input

POST /api/github/commit
6-Step Git API Flow
1. GET branch ref SHA
2. GET tree SHA
3. POST blob (base64)
4. POST new tree
5. POST commit
6. PATCH branch ref
Protected branches: 403
Zod: Conventional Commits

Webhooks
POST /api/webhooks/github
HMAC-SHA256 validated
timingSafeEqual compare
401 before any DB access
issues: opened → cache
issues: closed → done
PR: merged → close issue
push → staleness reset

Docs API
GET/PATCH /api/docs/[id]
Supabase cache (30 min TTL)
Notion pull on stale/miss
GitHub snapshot on archive

Cron Jobs
/api/cron/nightly 2:00am
Staleness check (5d+)
/api/cron/morning 8:00am
Standup digest → doc

Zoom API Routes
/api/zoom/meetings
OBF/ZAK Token Auth
Webhook signature verified
No recordings stored

Delete File API
POST /api/github/delete-file
7-step flow: quarantine first
Confirm token required
Audit log + recovery cmd

DATA LAYER — Supabase (PostgreSQL + RLS)

Supabase (SOC2 Type II)
workspace_id on EVERY table · RLS on EVERY query

workspaces workspace_id PK	profiles role, workspace_id	issues (cache) urgency, sprint_id
teams workspace scoped	docs (cache) notion TTL 30min	activity_feed audit + events

🔒 workspace_isolation RLS policy on every table
USING (workspace_id = (SELECT workspace_id FROM profiles WHERE id = auth.uid(

Realtime / Pusher
workspace-{workspaceId}-{purpose}
Cloud: Pusher.com (dev / early prod)
Self-hosted: Soketi (Docker)
PUSHER_HOST=soketi, PUSHER_PORT=6001
Events: issue_created · issue_status_changed
issue_assigned · pr_opened · sprint_updated
🔒 Channel prefix = tenant isolation
No workspace prefix = cross-tenant leak

EXTERNAL SERVICES

GitHub
SOC2 Type II · FedRAMP
Source of Truth (always)
OAuth Provider
Issues / PRs / Commits
Webhooks (HMAC-SHA256)
CodeQL + Secret Scanning
Dependabot auto-PRs
Branch protection: main

Zoom
SOC2 Type II · ISO 27001
REST API (meetings)
OBF / ZAK Token Auth
→ March 2026 required
Webhook secret validated
No recordings stored
Internal use only
Credentials: env vars only

Notion + Lovable + Vercel
Notion: Docs source (read-only)
page_id pull, 30-min cache
Lovable: App hosting + CI/CD
Deploy on push to main
Vercel: Cron job scheduler
nightly 2am · morning 8am
CRON_SECRET bearer auth

DEPLOYMENT OPTIONS

① Lovable Hosted (Current / Default)
Lovable URL = Live Production App
Supabase Cloud · Pusher Cloud · Vercel Crons
No deploy step · No server management
Best for: Dev + early production

② Docker Compose (Single Server)
app · postgres · realtime · kong
redis · soketi (Pusher-compatible)
MCP optional (sandboxed read-only)
Best for: Data residency · ~50 concurrent users

③ Kubernetes (Production Scale)
Next.js: Deployment 2-4 replicas · HPA CPU 70%
Postgres: StatefulSet or managed (RDS/Cloud SQL)
Soketi: sticky sessions · ingress-nginx + cert-manager
Best for: Multi-region · Large teams

VS Code Extension Distribution
VS Marketplace · Free · Open Source
Compatible: VS Code · Cursor · Windsurf
VSCodium · Gitpod · GitHub Codespaces
Same GitHub OAuth — no separate auth

SECURITY CONTROLS — Attack Surfaces Closed

🔒 **GitHub Token Never in Browser**
All calls via /api/github/proxy. provider_token extracted server-side only. DevTools = 0 tokens.

🔒 **workspace_id Never From Client**
requireWorkspaceId() reads from session profile. Client-supplied workspace_id = rejected (403).

🔒 **RLS = Last Line of Defense**
Postgres enforces at DB level. App bug cannot bypass. Defense-in-depth with app-level filter.

🔒 **SAST: GitHub CodeQL on Every PR**
CodeQL + Secret Scanning + Dependabot active. High/Critical alerts block merge. npm audit CI.

⚡ **Webhook Forgery Prevention**
HMAC-SHA256 on all GitHub webhooks. timingSafeEqual. 401 before any DB access.

⚡ **Pusher Channel Tenant Isolation**
Every channel: workspace-{workspaceId}-{purpose}. No prefix = cross-tenant event leak (blocked).

⚡ **SSRF Prevention in GitHub Proxy**
endpoint field must start with /. Absolute URLs rejected. Prevents internal network access.

⚡ **Zoom: OBF/ZAK + No Recordings**
OBF/ZAK tokens per Zoom March 2026 mandate. Webhook signature verified. Zero recordings stored.

🔐 All data in transit: TLS 1.2+ · All data at rest: AES-256 (Supabase) · Secrets: Env vars only, never in git · Sessions: GitHub OAuth, time-limited tokens